

Lộ trình 9 tuần cho Dự án E-Commerce System (4 người)

Phân công theo Layer

Layer	Người	Nhiệm vụ chính
Model & Util	Thành viên 1	Entity classes, enums, helpers, DatabaseConnection
Repository	Thành viên 2	JDBC, SQL, CRUD, JOIN queries, data access
Service	Thành viên 3	Business logic, validation, billing computation, role check
UI	Thành viên 4	Java Swing, JTable, JOptionPane, event handling

Tips cho nhóm 4 người

1. **Dùng Git branches:** mỗi người 1 branch, merge về main mỗi tuần
2. **Code convention:** Thống nhất naming, format code từ đầu
3. **Shared Google Drive:** Để tài liệu, test data
4. **Backup plan:** Nếu ai đó bận, người khác cover được
5. **Early integration:** Đừng đợi tuần 8 mới ghép code
6. **Reuse code:** Share utility classes (FileHandler, Validator, etc.)

TUẦN 1: Setup & Design (CẢ NHÓM)

Cùng họp và làm:

- Vẽ Class Diagram đầy đủ
- Thiết kế ERD (Entity Relationship Diagram) cho database
- Định nghĩa Database Schema (tables, columns, relationships)
- Setup Git repository + branching strategy
- Tạo folder structure theo layer:

```
src/  
├── model/      (Thành viên 1)  
└── repository/ (Thành viên 2)
```

```
└─ service/      (Thành viên 3)
   └─ ui/         (Thành viên 4)
```

- Thống nhất naming convention và coding standards

Deliverable: Document thiết kế + ERD + Class Diagram

TUẦN 2: Model & Database Foundation 🏠

Thành viên 1 (model & util):

- Tạo các entity classes:
 - `User` (username, password, role)
 - `Customer` (id, name, phone, email, createdAt)
 - `Item` (sku, name, price, category)
 - `Order` (orderId, customerId, orderDate, status, totalAmount)
 - `OrderItem` (orderId, itemSku, quantity, price)
 - `Coupon` (code, discountType, discountValue, expiryDate)
- Tạo `DatabaseConnection` utility class
- Tạo các helper classes (Validator, DateFormatter, etc.)

Thành viên 2 (repository):

- Setup database connection với JDBC
- Tạo SQL scripts để tạo tables
- Implement interface pattern:

```
interface UserRepository { ... }
interface CustomerRepository { ... }
interface ItemRepository { ... }
```

- Test connection và basic CRUD với dummy data

Thành viên 3 (service):

- Định nghĩa các Service interfaces:

```
interface AuthenticationService { ... }
interface CustomerService { ... }
```

```
interface ItemService { ... }
```

- Chuẩn bị validation rules

Thành viên 4 (ui):

- Research Java Swing components
- Tạo mockup/wireframe cho các màn hình
- Setup JFrame template và color scheme

Checkpoint: Review ERD, test database connection

TUẦN 3: Repository Layer Complete

Thành viên 1 (model):

- Thêm constructor, getters/setters cho tất cả entities
- Implement `toString()` và `equals()` methods
- Tạo enums: `UserRole`, `OrderStatus`, `DiscountType`

Thành viên 2 (repository) - FOCUS WEEK:

Implement tất cả Repository classes:

- `UserRepositoryImpl` - CRUD User + authentication queries
- `CustomerRepositoryImpl` - CRUD + search by name/phone
- `ItemRepositoryImpl` - CRUD + check duplicate SKU
- `OrderRepositoryImpl` - CRUD + complex JOIN queries
- `OrderItemRepositoryImpl` - Manage order items
- `CouponRepositoryImpl` - CRUD + validate expiry

Viết SQL queries tối ưu với JOIN:

```
-- Example: Get order details with customer info
SELECT o.*, c.name, c.phone
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.order_id = ?
```

Thành viên 3 (service):

- Bắt đầu implement Service layer (gọi Repository)
- Validation logic (email format, phone format)

Thành viên 4 (ui):

- Tạo Login screen prototype
- Tạo Main window với JTabbedPane template

Test: Repository layer với unit tests

TUẦN 4: Service Layer - Business Logic 🧠

Thành viên 1 (model):

- Support Thành viên 3: tạo thêm utility classes nếu cần
- Code review cho Repository layer

Thành viên 2 (repository):

- Optimize queries, add indexes
- Implement advanced search/filter queries
- Support Thành viên 3

Thành viên 3 (service) - FOCUS WEEK:

Implement tất cả Service classes:

AuthenticationService:

- `login(username, password)` - validate credentials
- `logout()` - record logout event
- `checkPermission(role, action)` - role-based access

CustomerService:

- CRUD operations với validation
- `searchCustomer(keyword)`
- `getCustomerOrderHistory(customerId)`

ItemService:

- CRUD với admin role check

- `updatePrice(sku, newPrice, adminUser)` - only Admin
- `checkSkuDuplicate(sku)`

OrderService:

- `createOrder(customerId, items, coupon)`
- `updateOrderStatus(orderId, status)`
- `cancelOrder(orderId)` - audit log

BillingService - CỰC KỲ QUAN TRỌNG:

```
// 3 overloaded methods
public double computeBill(double price) {
    return applyTax(price);
}

public double computeBill(double price, int quantity) {
    return applyTax(price * quantity);
}

public double computeBill(double price, int quantity, double couponDiscount) {
    double subtotal = price * quantity;
    double afterDiscount = subtotal - couponDiscount;
    return applyTax(afterDiscount);
}

private double applyTax(double amount) {
    return amount * 1.08; // 8% tax
}
```

CouponService:

- `validateCoupon(code)` - check expiry date
- `applyCoupon(orderTotal, couponCode)`

Thành viên 4 (ui):

- Học cách gọi Service layer từ UI
- Tạo JOptionPane templates cho errors

Test: Service layer với integration tests

TUẦN 5: UI Foundation 🎨

Thành viên 1 & 2:

- Support Thành viên 4
- Code review
- Prepare test data (100 customers, 50 items, 200 orders)

Thành viên 3:

- Hoàn thiện Dashboard statistics logic
- Audit log implementation

Thành viên 4 (ui) - FOCUS WEEK:

LoginFrame:

- Username/Password fields
- Login button → gọi `AuthenticationService.login()`
- Show error với `JOptionPane`
- Redirect to `MainFrame` nếu success

MainFrame:

- `JTabbedPane` với 5 tabs: Customers, Items, Orders, Billing, Dashboard
- Menu bar với Logout option
- Role-based tab visibility (Admin thấy hết, Staff ẩn một số)

CustomerPanel (Tab 1):

- `JTable` hiển thị danh sách customers
- Buttons: Add, Edit, Delete, Search
- Form panel (name, phone, email fields)
- Connect với `CustomerService`

Cơ bản:

```
// Load data
List<Customer> customers = customerService.getAllCustomers();
tableModel.setRowCount(0);
for(Customer c : customers) {
    tableModel.addRow(new Object[]{c.getId(), c.getName(), ...});
}
```

TUẦN 6: UI Advanced - Items & Orders 📁

Thành viên 1 & 2:

- Performance optimization
- Fix bugs từ testing

Thành viên 3:

- Support UI với complex queries
- Implement search/filter logic

Thành viên 4 (ui) - FOCUS WEEK:

ItemPanel (Tab 2):

- JTable items với columns: SKU, Name, Price, Category
- CRUD buttons
- Admin-only edit price (check role)
- Validation: SKU duplicate check

OrderPanel (Tab 3):

- JTable orders list
- Form tạo đơn hàng mới:
 - JComboBox chọn customer
 - JTable thêm items (item, quantity, price)
 - Button "Add Item to Order"
 - Display running subtotal
- Edit/Delete order buttons
- Order status JComboBox (Pending/Paid/Cancelled)

Complex UI logic:

```
// Add item to order
JButton btnAddItem = new JButton("Add Item");
btnAddItem.addActionListener(e -> {
    String sku = cmbItem.getSelectedItem();
    int qty = (int)spnQuantity.getValue();
    // Add to table, update subtotal
});
```

TUẦN 7: Billing & Dashboard

Thành viên 1 & 2:

- Database optimization
- Backup/restore functionality

Thành viên 3:

- Fine-tune billing calculations
- Test edge cases

Thành viên 4 (ui) - FOCUS WEEK:

BillingPanel (Tab 4):

- Input fields: Price, Quantity, Coupon code
- **Real-time calculation display:**

```
Subtotal: $100.00
Discount: -$10.00
Tax (8%): +$7.20
Total: $97.20
```

- Buttons to test 3 overloaded methods:
 - "Calculate (Price only)"
 - "Calculate (Price + Qty)"
 - "Calculate (Full with Coupon)"
- "Generate Invoice" button → text area hoặc print dialog

Invoice format:

```
===== INVOICE =====
Order ID: ORD001
Customer: John Doe
Date: 2026-01-23

-----
Item      Qty    Price
Product A    2    $50.00
-----

Subtotal:           $100.00
Discount (10%):     -$10.00
```

Tax (8%):	+\$7.20
=====	
TOTAL:	\$97.20

DashboardPanel (Tab 5):

- Statistics cards (JPanel với labels):
 - Total Orders
 - Total Revenue
 - Cancelled Orders
- JTable orders với sort headers
- Filter panel:
 - JComboBox status filter
 - JDateChooser date range
 - Search field (customer name/order ID)
- "Refresh" button

Implementation:

```
// Real-time billing update
txtQuantity.addKeyListener(new KeyAdapter() {
    public void keyReleased(KeyEvent e) {
        updateBillingDisplay();
    }
});

void updateBillingDisplay() {
    double price = Double.parseDouble(txtPrice.getText());
    int qty = Integer.parseInt(txtQuantity.getText());
    String coupon = txtCoupon.getText();

    double total = billingService.computeBill(price, qty,
        couponService.getDiscountAmount(coupon));
    lblTotal.setText("$" + total);
}
```

TUẦN 8: Integration & Testing

CẢ NHÓM cùng làm:

Ngày 1-2: Integration

- **Thành viên 1:** Ensure models work với tất cả layers
- **Thành viên 2:** Check database integrity, transactions
- **Thành viên 3:** End-to-end service testing
- **Thành viên 4:** Connect UI hoàn chỉnh với backend

Complete workflow test:

1. Login as Admin
2. Add 5 customers
3. Add 10 items
4. Create 20 orders
5. Apply coupons, change status
6. View dashboard statistics
7. Search/filter orders
8. Generate invoices
9. Logout

Ngày 3-4: Bug Fixing Sprint

- Mỗi người test chéo module của người khác
- Fix critical bugs
- Handle edge cases:
 - Empty database
 - Invalid inputs
 - Expired coupons
 - Duplicate SKU
 - Division by zero

Ngày 5-7: Polish & Optimization

- **Thành viên 1:** Optimize entity classes, add JavaDoc
- **Thành viên 2:** Database indexing, query optimization
- **Thành viên 3:** Service layer refactoring
- **Thành viên 4:** UI/UX improvements (colors, icons, layout)

TUẦN 9: Finalization & Delivery

Ngày 1-3: Testing & Documentation

Thành viên 1:

- Unit tests cho model classes
- Generate JavaDoc
- Class Diagram finalization

Thành viên 2:

- Database documentation (ERD, table schemas)
- SQL script file cho setup
- Repository layer testing

Thành viên 3:

- Service layer documentation
- Business logic flowcharts
- Test các tính năng billing kỹ lưỡng

Thành viên 4:

- User Manual (screenshots + hướng dẫn)
- UI testing checklist
- Video demo recording (5-10 phút)

Ngày 4-5: Presentation Prep

- **Cả nhóm:** Làm slide (20-25 slides)
 - Overview
 - Architecture (4 layers)
 - Database design
 - Key features demo
 - Challenges & solutions
 - Q&A prep
- Rehearsal 2-3 lần

Ngày 6: Final Review

- Code cleanup
- Remove debug code
- Final testing
- Package deliverables:
 - Source code (ZIP)

- Database script
- User manual (PDF)
- Presentation (PPT)
- Demo video

Ngày 7: SUBMISSION 🎉

📞 Communication Flow

```
UI (Thành viên 4)
  ↓ gọi
Service (Thành viên 3)
  ↓ gọi
Repository (Thành viên 2)
  ↓ truy vấn
Database / Model (Thành viên 1)
```

Ví dụ flow tạo đơn hàng:

1. User nhấn "Create Order" button (UI)
 2. `OrderPanel` gọi `orderService.createOrder()` (Service)
 3. Service validate, tính toán, gọi `orderRepository.save()` (Repository)
 4. Repository execute SQL INSERT với model `Order` (Model + DB)
 5. Return success → UI hiển thị message
-

⚠️ Lưu ý quan trọng cho nhóm

Dependencies giữa các layer:

- **Tuần 2:** Thành viên 1 phải hoàn thành Model trước → Thành viên 2 mới code Repository
- **Tuần 3:** Thành viên 2 phải hoàn thành Repository → Thành viên 3 mới code Service
- **Tuần 4:** Thành viên 3 phải hoàn thành Service → Thành viên 4 mới kết nối UI

Giải pháp:

- Thành viên 4 có thể làm UI mockup/layout từ tuần 2-3 (chưa cần connect backend)
- Thành viên 1 support các thành viên khác khi xong sớm
- Daily standup để sync tiến độ

Git workflow:

```
main (production)
├── feature/model (Thành viên 1)
├── feature/repository (Thành viên 2)
├── feature/service (Thành viên 3)
└── feature/ui (Thành viên 4)
```

Merge về main mỗi cuối tuần sau khi review code.



Meeting Schedule

- **Thứ 2:** Planning meeting (30p) - review tiến độ tuần trước, plan tuần này
 - **Thứ 6:** Code review (1-2h) - merge code, fix conflicts, test
 - **Group chat:** Daily standup report (5p mỗi người báo cáo làm được gì)
-



Milestones quan trọng

- ☒ **Tuần 3:** Backend core hoàn thành
 - ☒ **Tuần 5:** Billing logic + Order management xong
 - ☒ **Tuần 7:** GUI cơ bản hoạt động được
 - ☒ **Tuần 8:** Hệ thống chạy được end-to-end
 - ☒ **Tuần 9:** Sẵn sàng demo
-